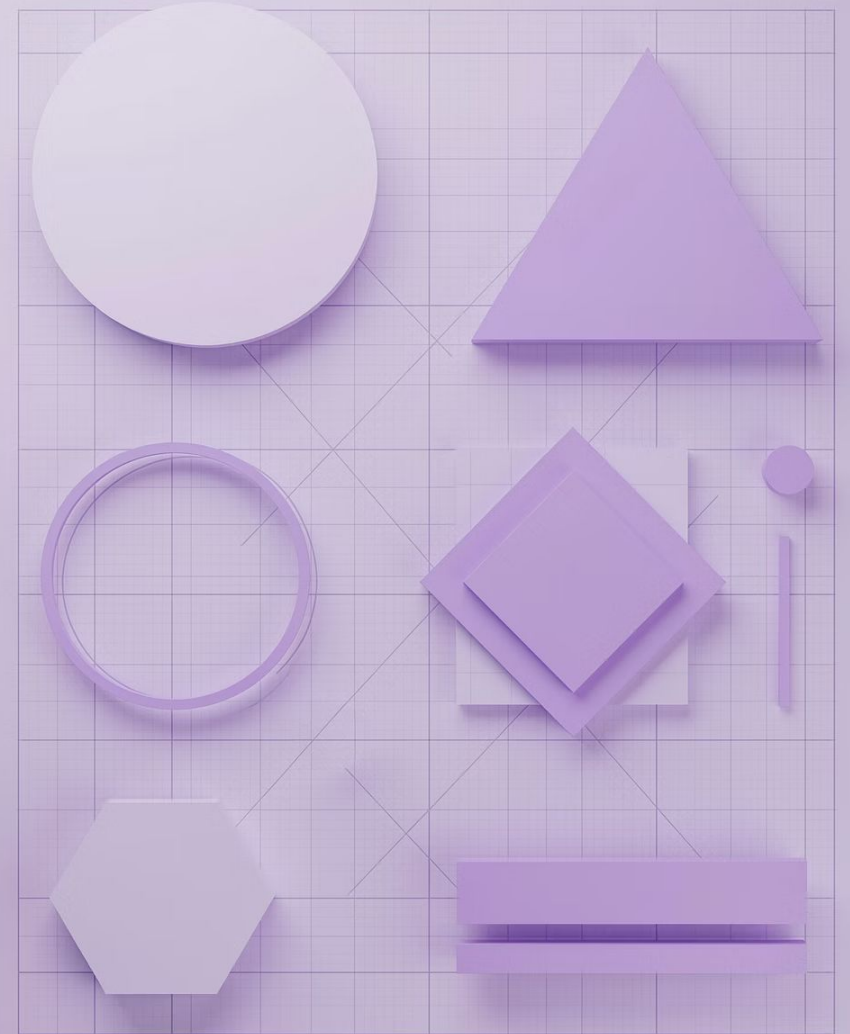


PROGRAMACIÓN ORIENTADA A OBJETOS

Clase asociación

Cuando la relación misma tiene datos propios

No toda conexión entre clases es simple: a veces, **la relación en sí porta información** que no pertenece a ninguna de las dos partes por separado.



¿Qué es una clase asociación?

Una **asociación normal** solo conecta dos clases. Se convierte en **clase asociación** cuando la relación en sí tiene un atributo que **no pertenece a ninguna de las dos clases por separado**, sino a la **combinación de ambas**.

El dato en cuestión no describe a uno solo de los objetos relacionados: describe al vínculo. Si intentás asignarlo a una sola clase, perdés información esencial o introducís redundancia. Ese dato vive en la **relación misma**, no en ninguno de sus extremos.

El test de reconocimiento

"¿Este dato depende del **PAR de objetos**, y no de uno solo de ellos?"

- **Sí** → es clase asociación → reificala como nueva clase
- **No** → el dato pertenece a una de las dos clases

Asociación simple

Solo conecta dos clases. No hay datos en la relación. No se reifica.

Clase asociación

La relación tiene atributos propios. Se convierte en una **nueva clase** que reifica el vínculo.

La señal en el enunciado

Palabras clave: "**registra**", "**para cada combinación**", "**en ese contexto**", "**del par**".

EJEMPLO 1

Restaurante ofrece Plato → PlatoServido

Este ejemplo muestra cómo un dato que parece pertenecer a una clase termina siendo, en realidad, un dato de la **relación entre dos clases**.



Enunciado

"Un restaurante ofrece hasta 20 platos, y para cada plato que ofrece **registra una valoración**."



¿De quién es la valoración?

No es del **Restaurante** (no es igual para todos sus platos) ni del **Plato** (varía según en qué restaurante se sirva). Es de la combinación "**este plato en este restaurante**".



La reificación

El vínculo se convierte en una nueva clase: **PlatoServido**. Tiene la valoración como atributo propio, y conecta a Restaurante con Plato.

Instancias distintas, relaciones distintas — y nuevas conexiones posibles

Reificar la relación no es solo un trámite formal: habilita modelado más preciso y abre la puerta a **nuevas asociaciones** sobre la clase reificada.

Cada par genera su propia instancia

La misma **Milanesa** puede tener valoración **9** en un restaurante y valoración **4** en otro. Esos son dos **PlatoServido distintos**, no uno compartido.

Si la valoración estuviera en la clase Plato, no habría forma de registrar que el mismo plato tiene calidades distintas en distintos contextos. La clase asociación resuelve exactamente eso.

La clase reificada entra en otras relaciones

→ No le gusta el Plato abstracto

"A una persona le gusta el **PlatoServido**" — no el plato en general, sino cómo lo sirven en un restaurante concreto.

→ La reificación habilita nuevas asociaciones

La clase **PlatoServido** puede asociarse con otras clases (Persona, Pedido, etc.) igual que cualquier clase normal.

Dos clases asociación a la vez: Avión, Propietario y Mecánico

A veces un mismo modelo tiene **más de una clase asociación**, cada una con su propia cardinalidad y sus propias reglas.

Enunciado: "Un avión tiene un historial de propietarios (uno vigente por vez).
Un mecánico registra servicios sobre un avión, pudiendo repetir avión y mecánico siempre que no se repita fecha y tipo de trabajo."

1 Propiedad (Avión–Persona)

La relación tiene datos propios (fechas, condición de vigencia).
Cardinalidad acotada: **una vigente por vez**, pero con historial de anteriores.

2 Servicio (Mecánico–Avión)

La relación tiene datos propios (fecha, tipo de trabajo).
Cardinalidad libre: se puede repetir el par mecánico-avión, con una **regla de unicidad** propia sobre fecha + tipo.

¿Qué distingue a cada una?

- **Propiedad:** una vigente, pero el historial importa → la cardinalidad está regulada por estado
- **Servicio:** muchas posibles, sin límite → solo hay restricción de duplicado exacto

El patrón base es el mismo — la relación tiene datos — pero las **reglas de cardinalidad** son completamente distintas.

Tipo vs. instancia: el clasificador compartido

No todo dato que "parece" de un objeto es realmente de ese objeto. A veces pertenece a una **categoría compartida**, no a la instancia individual.



Enunciado

"Todos los aviones de modelo **B737** comparten la misma capacidad y el mismo peso."



El dato no es del avión individual

La capacidad y el peso no cambian de avión a avión dentro del mismo modelo. Son datos del **tipo**, no de la instancia. Se modelan en una clase **TipoAvion**.



Las autorizaciones también son por tipo

Los pilotos y mecánicos no se autorizan avión por avión: se autorizan por **tipo de avión**. La habilitación pertenece al clasificador, no a cada instancia.

¿El dato es igual para todos, o cambia de instancia a instancia?

Ante cualquier atributo que parezca pertenecer a un objeto, aplicá esta pregunta antes de modelarlo:

"¿Es igual para TODOS los de esa categoría?"

Si todos los B737 tienen la misma capacidad → el dato es del **tipo**.
Sepáralo en una clase Tipo aparte y asocia.

"¿Cambia de instancia a instancia?"

Si cada avión individual tiene su propio valor → el dato es un **atributo de la instancia**. Va directo en la clase.

"¿Aplica a la combinación?"

Si el dato depende del **par de objetos** → es clase asociación.
Reifícala como nueva clase.

Si ponés en el avión individual un dato que es del tipo, lo estás **repitiendo en cada instancia**. Cualquier cambio exige actualizar todos los objetos. El modelo empieza a ser inconsistente.



La trampa a recordar: el patrón base es el mismo, los detalles no

Reconocer una clase asociación es solo el primer paso. Lo que cambia el modelo real son la **cardinalidad** y el **nivel** al que pertenece el dato.



La cardinalidad importa

Una vigente con historial

Como Propiedad: hay una relación activa, pero las anteriores se conservan. No es un simple reemplazo.

Muchas libres con unicidad

Como Servicio: el par puede repetirse, pero no el triplete (par + fecha + tipo). La restricción es más fina.

Libre sin restricción extra

Como PlatoServido: cada combinación restaurante-plato genera su propia instancia, sin límite adicional.



El nivel también importa

Dato de instancia

Varía de objeto a objeto. Va como atributo de la clase individual o de la clase asociación.

Dato de tipo

Igual para todos los objetos de esa categoría. Va en una clase clasificadora separada (TipoAvion).

La base siempre es la misma

"La relación tiene datos propios" → reificala. Lo que varía es **cómo** modelar esa reificación.

RESUMEN

Cuadro comparativo: clases asociación y clasificador

Ante cada relación del modelo, aplicá el test. Si el dato depende del par → clase asociación. Si es igual para todos los de una categoría → clasificador separado.

 Ante cada relación del modelo, aplicá el test. Si el dato depende del par → clase asociación. Si es igual para todos de una categoría → clasificador separado.

Test 1: Depende del par?

Si el dato depende del par de objetos → clase asociación (reificar).

Test 2: Igual para todos?

Si es igual para todos de la categoría → clasificador separado (TipoAvion).

No en ambos

Si no aplica ninguno → atributo de instancia normal.

Clase	Clases que relaciona	Cardinalidad	Notas clave
PlatoServido	Restaurante – Plato	Libre (muchos por restaurante)	La valoración depende del par. Puede entrar en otras relaciones (le gusta a Persona).
Propiedad	Avión – Persona	Una vigente + historial	La relación activa es única, pero las anteriores se conservan. Estado "vigente" como atributo.
Servicio	Mecánico – Avión	Libre con unicidad propia	Se puede repetir el par mecánico-avión; la restricción es fecha + tipo de trabajo.
TipoAvion	Clasificador de Avión	Compartido por todas las instancias del tipo	<i>No es clase asociación:</i> es un clasificador. Capacidad, peso y autorizaciones viven aquí.